

Monitoramento Contínuo de Performance

Solução para e-commerce com Prometheus, Grafana, Alertmanager, JMeter e Locust

Identificação

Aluno: Kevin Maikon Caetano de Andrade Santos

Universidade: UnifECAP

Disciplina: Quality Assurance

Data: 30 de maio de 2026

Introdução: contexto do desafio

PROBLEMA

A E-Fecaf Global é um e-commerce em expansão que sofre com lentidão, quedas e perda de vendas em picos de acesso.

Impactos identificados

- Lentidão em páginas e checkout.
- Quedas durante campanhas promocionais.
- Abandono de carrinho e perda de receita.
- Reclamações em canais digitais.
- Dificuldade para encontrar a causa raiz.

Objetivo da solução

- Implantar monitoramento contínuo.
- Visualizar indicadores em tempo real.
- Configurar alertas automáticos.
- Executar testes de estresse.
- Otimizar a plataforma com base em dados.

Principais desafios

DIAGNÓSTICO

Os desafios se conectam: sem visibilidade e alertas, a equipe responde tarde e o impacto no negócio aumenta.

Lentidão

- Páginas lentas.
- Checkout com atrasos.
- APIs degradadas em pico.

Indisponibilidade

- Quedas em promoções.
- Compra interrompida.
- Perda de confiança.

Falta de visibilidade

- Métricas descentralizadas.
- Gargalos ocultos.
- Baixa correlação de falhas.

Falta de alertas

- Incidentes descobertos tarde.
- Recuperação lenta.
- Risco em datas críticas.

Arquitetura de monitoramento contínuo

SOLUÇÃO PROPOSTA

A proposta combina coleta, dashboards, alertas e testes para criar um ciclo de melhoria permanente.

Aplicações, APIs,
servidores e banco
expõem métricas



Prometheus coleta
e armazena séries
temporais



Grafana visualiza
e Alertmanager
notifica equipes

JMeter e Locust

Geram carga controlada para validar a capacidade em cenários como Black Friday.

Ferramentas propostas

FERRAMENTAS

Cada ferramenta possui uma função específica na solução de observabilidade e testes.



Prometheus

Coleta e armazena métricas em séries temporais.



Grafana

Cria dashboards executivos e técnicos.



Alertmanager

Envia alertas automáticos e escalonados.



JMeter

Executa testes estruturados de carga.



Locust

Simula usuários com cenários em Python.

Resultado esperado

Maior previsibilidade, identificação rápida de gargalos, redução de incidentes e melhoria da experiência do cliente.

Prometheus

Núcleo da solução para coletar e consultar métricas técnicas e de negócio em tempo real.

Como funciona

- Modelo pull para coleta de métricas.
- Consulta endpoints HTTP de aplicações e exporters.
- Armazena dados como séries temporais.
- Permite análise com PromQL.
- Integra-se ao Alertmanager.

Benefícios

- Monitoramento em tempo real.
- Flexibilidade para métricas técnicas e de negócio.
- Identificação rápida de anomalias.
- Base sólida para melhoria contínua.
- Exemplo: `http_request_duration_seconds`.

Grafana: dashboards em tempo real

VISUALIZAÇÃO

Transforma métricas em painéis claros para equipes técnicas, gestores e áreas de negócio.

Disponibilidade

99,4%

Resposta média

1,6s

Erros 5xx

2,1%

Requisições/min

42k

CPU média

71%

Alertas ativos

3

Alertmanager

ALERTAS AUTOMÁTICOS

Recebe alertas do Prometheus, organiza notificações e reduz o tempo de resposta a incidentes.

Canais de notificação

- E-mail corporativo.
- Microsoft Teams, Slack ou Google Chat.
- Sistemas de chamados.
- Ferramentas de plantão.
- Escalonamento para DevOps e liderança.

Fluxo de resposta

- Prometheus detecta condição crítica.
- Alertmanager agrupa e classifica.
- Equipe recebe alerta acionável.
- Responsáveis executam correção.
- Incidentes graves são escalonados.

Métricas essenciais monitoradas

As métricas foram escolhidas por relação direta com performance, disponibilidade e experiência do cliente.

Métrica	Objetivo	Exemplo de uso
Tempo de resposta	Medir velocidade percebida pelo usuário.	Checkout deve responder em até 2 segundos.
Latência	Avaliar atraso em APIs e integrações.	Identificar APIs lentas em horários de pico.
Disponibilidade	Controlar estabilidade do e-commerce.	Manter disponibilidade acima de 99%.
CPU e memória	Medir carga e consumo dos serviços.	Prever escala ou investigar vazamentos.
Taxa de erros	Medir falhas em requisições.	Alertar quando erros superarem 5%.

Configuração de alertas

ALERTAS

Bons alertas indicam o que está acontecendo, onde está o problema e qual é a severidade.

Condição	Severidade	Ação recomendada
CPU acima de 85% por 5 minutos	Alta	Verificar saturação e escalar servidores.
Memória acima de 90% por 5 minutos	Alta	Investigar vazamento ou aumentar recursos.
Tempo de resposta acima de 2 segundos	Crítica	Avaliar APIs, banco de dados e cache.
Disponibilidade abaixo de 99%	Crítica	Acionar plano de incidente.
Taxa de erro acima de 5%	Crítica	Identificar serviço com falha e corrigir.

Exemplo de regra Prometheus

PROMQL

A regra abaixo demonstra um alerta de alto uso de CPU encaminhado ao Alertmanager.

```
groups:
  - name: e-fecaf-alertas
    rules:
      - alert: AltoUsoCPU
        expr: avg(rate(node_cpu_seconds_total{mode!="idle"}[5m])) * 100 > 85
        for: 5m
        labels:
          severity: alta
        annotations:
          summary: "Uso de CPU acima de 85%"
```

Como isso ajuda

- Detecção automática de risco.
- Alerta só dispara após 5 minutos.
- Reduz ruído operacional.
- Notificação chega aos responsáveis.
- Permite ação antes do impacto ao cliente.

Testes de estresse com JMeter

JMETER

Ferramenta para testes de carga, estresse e performance em aplicações web, APIs e serviços.

1 Cenários

Ex: login, busca, produto, carrinho, checkout e pagamento.

2 Usuários

Configurar usuários virtuais simultâneos.

3 Ramp-up

Definir crescimento gradual de carga.

4 Execução

Executar em ambiente controlado.

5 Análise

Visualizar resposta, throughput e erros.

Testes de estresse com Locust

LOCUST

Complementa o JMeter com cenários programáveis em Python e execução distribuída.

Vantagens

- Cenários escritos em Python.
- Comportamento real de usuários.
- Execução distribuída para altos volumes.
- Interface web em tempo real.
- Boa integração com pipelines DevOps.

Critério	JMeter	Locust
Abordagem	Planos visuais	Código Python
Facilidade	Boa inicial	Boa com Python
Escalabilidade	Boa	Muito boa
Cenários	Visual	Mais flexível

Simulação de Black Friday

CENÁRIO DE TESTE

A carga cresce progressivamente para validar estabilidade, checkout, APIs, banco de dados e autoscaling.



Resultados esperados

- Disponibilidade acima de 99%.
- Tempo médio abaixo de 2 segundos.
- Taxa de erro abaixo de 5%.
- Checkout funcional durante todo o teste.
- Gargalos identificados com evidência.

Gargalos plausíveis encontrados

ANÁLISE

Monitoramento e testes permitem apontar gargalos com base em dados, não em suposições.

Banco de dados

- Consultas sem índice.
- Busca lenta.
- Bloqueios em pedidos.

Servidores

- CPU acima de 85%.
- Memória acima de 90%.
- Escala insuficiente.

APIs

- Checkout acima de 2s.
- Erros 500 em pagamento.
- Conexões limitadas.

Rede

- Serviços externos lentos.
- Ausência de CDN.
- Gargalo no balanceador.

Propostas de otimização

MELHORIA CONTÍNUA

As ações devem ser priorizadas por impacto no checkout, pagamento, disponibilidade e receita.

Balanceamento

Distribuir requisições e evitar sobrecarga em uma instância.

Cache

Reduzir consultas repetidas e acelerar páginas e produtos.

SQL otimizado

Criar índices, revisar queries lentas e reduzir bloqueios.

Escala horizontal

Adicionar instâncias automaticamente em picos.

CDN

Entregar imagens e arquivos estáticos com menor latência.

Revisão contínua

Ajustar SLAs, alertas e capacidade com base no histórico.

Benefícios esperados

RESULTADOS

A empresa passa de uma operação reativa para uma operação preventiva, orientada por dados.

Disponibilidade

+99%

Meta de estabilidade.

Resposta

<2s

Meta para checkout.

Erros

<5%

Controle de falhas.

Recuperação

MTTR

Resposta mais rápida.

Valor para o cliente

Site mais rápido, checkout confiável e menos frustração em promoções.

Valor para o negócio

Menor abandono de carrinho, aumento de vendas e confiança na marca.

Conclusão e referências

ENCERRAMENTO

A solução integra monitoramento, visualização, alertas e testes para garantir estabilidade em picos de tráfego.

Resumo da solução

Prometheus coleta métricas; Grafana exibe dashboards; Alertmanager notifica equipes; JMeter e Locust validam capacidade.

Conclusão

Monitoramento contínuo é um processo permanente de prevenção de falhas, melhoria da experiência e proteção da receita.

Referências bibliográficas

- Apache JMeter User Manual - <https://jmeter.apache.org/usermanual/>
- Grafana Documentation - <https://grafana.com/docs/grafana/latest/>
- Locust Documentation - <https://docs.locust.io/>
- Prometheus Documentation - <https://prometheus.io/docs/>
- Alertmanager Documentation - <https://prometheus.io/docs/alerting/latest/alertmanager/>
- GOOGLE. Site Reliability Engineering. O'Reilly Media, 2016.